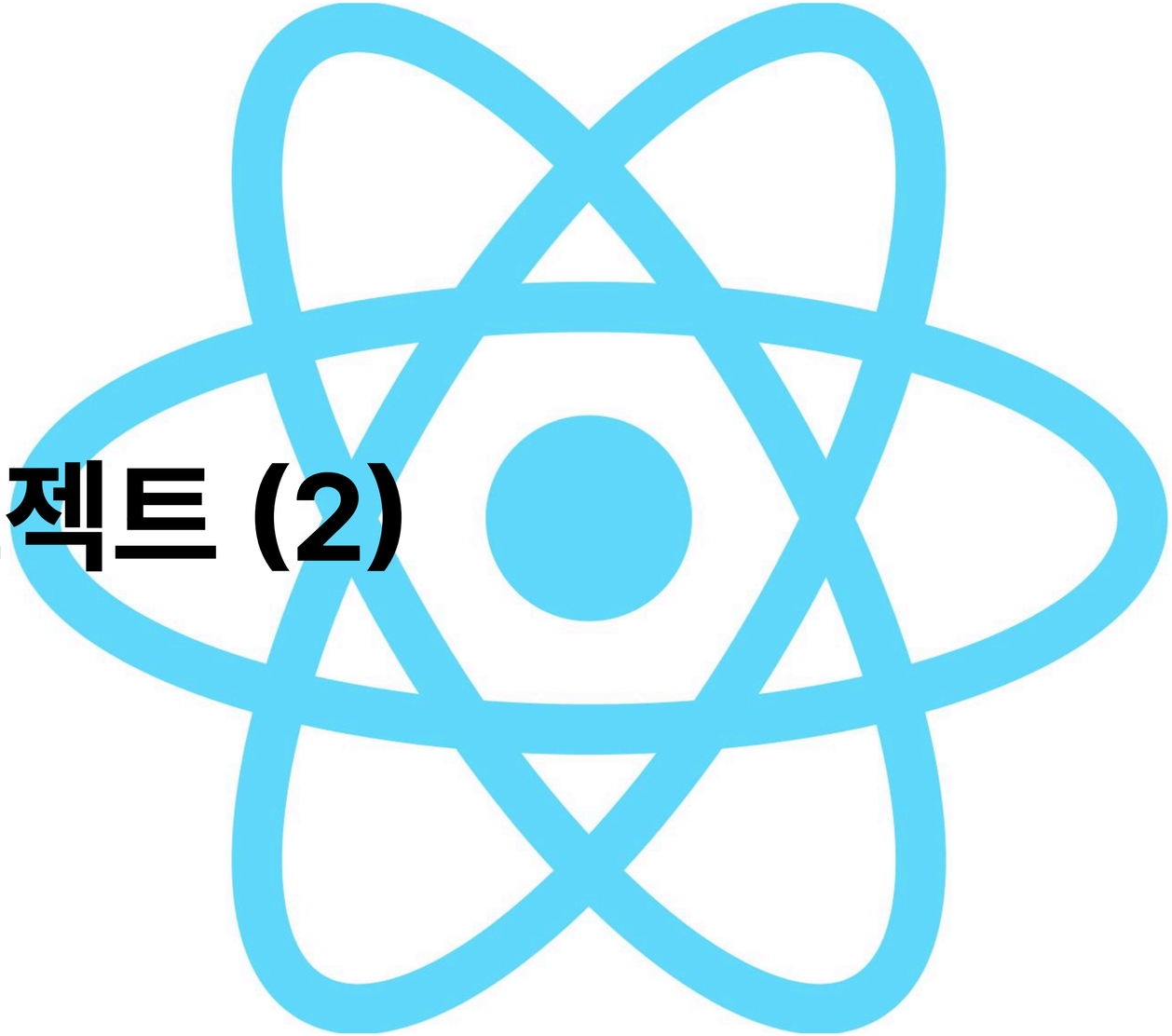
React 쇼핑몰 프로젝트 (2)



EY 기업맞춤형 AI+X 융복합과정 보조강사 정승민

2일차 목차

01 **1교시 장바구니 만들기**

상품 담기, 수량 변경, 삭제와 총 수량과 합계 계산

02 **2교시 주문서 작성과 DB 준비**

입력 검증과 PostgreSQL 상품 테이블과 데이터 준비

03 **3교시 FastAPI 상품 API 만들기**

HTTP 요청과 응답과 DB 상품 조회 API 구현

04 **4교시 React와 상품 API 연결하기**

Hooks: 비동기 요청 이해와 로딩, 오류 화면 처리

실습 코드

App.css 전체 교체

src/App.css: 기존 내용 전체를 아래 코드로 교체

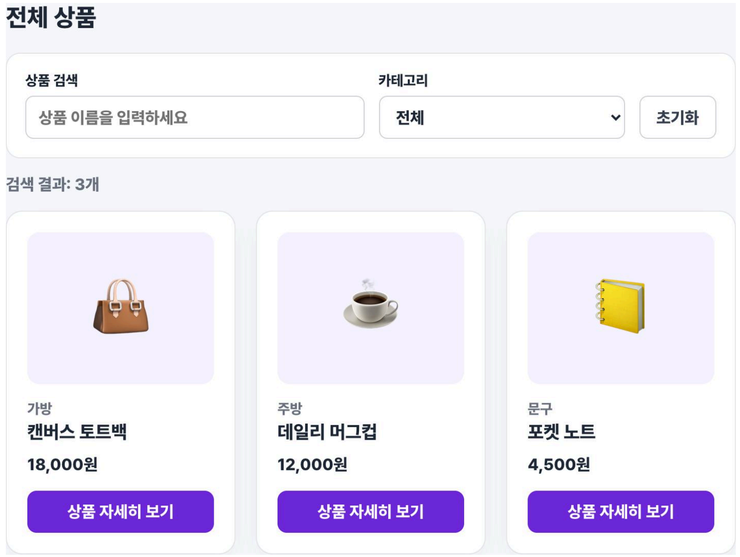
```
#root{width:100%; min-height:100vh;} .shop{width:min(100%, 1120px); margin:0 auto; padding:48px 24px 80px;} .shop-header{margin-bottom:48px; text-align:center;} .shop-header p{margin:0 0 8px; color:#7c3aed; font-weight:700;} .shop-header h1{margin:0 0 10px; color:#1f2937; font-size:clamp(32px, 4vw, 48px); line-height:1.2;} .shop-header span{color:#6b7280;} .shop h2{margin:0 0 20px; color:#1f2937; font-size:24px;} .product-grid{display:grid; grid-template-columns:repeat(auto-fit, minmax(220px, 1fr)); gap:20px;} .product-card{display:flex; min-height:320px; flex-direction:column; padding:20px; border:1px solid #e5e7eb; border-radius:16px; background:#fff; box-shadow:0 8px 24px rgba(31, 41, 55, 0.04);} .product-emoji{display:grid; height:148px; place-items:center; border-radius:12px; background:#f5f3ff; font-size:56px;} .product-category{margin:16px 0 4px; color:#6b7280; font-size:14px;} .product-card h3{margin:0 0 12px; font-size:18px;} .product-price{margin:auto 0 16px; font-weight:700;} .product-card button{width:100%; padding:11px 16px; border:0; border-radius:9px; background:#6d28d9; color:#fff; font-weight:700;} .product-card button:hover{background:#5b21b6;} @media (max-width: 640px){.shop{padding:28px 16px 56px;} .shop-header{margin-bottom:36px;} .product-grid{grid-template-columns:1fr;}} :root{font-family:"Pretendard", "Noto Sans KR", system-ui, -apple-system, sans-serif; color:#1f2937; background:#f7f8fc; font-synthesis:none; text-rendering:optimizeLegibility; -webkit-font-smoothing:antialiased;} *{box-sizing:border-box;} body{min-width:320px; margin:0;} button, input, select{font:inherit;} button{cursor:pointer;} .featured-section{margin-bottom:48px;} .featured-product{display:grid; width:100%; grid-template-columns:80px minmax(0, 1fr) auto auto; align-items:center; gap:18px; padding:20px 24px; border:1px solid #e6e2f0; border-radius:16px; background:#fff; box-shadow:0 8px 24px rgba(31, 41, 55, 0.05);} .featured-product div[role="img"]{display:grid; height:80px; place-items:center; border-radius:12px; background:#f5f3ff; font-size:42px;} .featured-product h3{margin:0; font-size:18px;} .featured-product p{margin:0; font-weight:700;} .featured-product button{padding:11px 16px; border:0; border-radius:9px; background:#6d28d9; color:#fff; font-weight:700; white-space:nowrap;} .featured-product button:hover{background:#5b21b6;} .filter-panel{display:grid; grid-template-columns:minmax(0, 1fr) minmax(180px, 240px) auto; align-items:end; gap:14px; margin-bottom:16px; padding:18px; border:1px solid #e5e7eb; border-radius:14px; background:#fff;} .filter-field{display:flex; min-width:0; flex-direction:column; gap:7px;} .filter-field label{font-size:14px; font-weight:700;} .filter-field input, .filter-field select{width:100%; min-height:42px; padding:9px 12px; border:1px solid #d1d5db; border-radius:8px; background:#fff; color:#1f2937;} .filter-field input:focus, .filter-field select:focus{outline:2px solid #a78bfa; outline-offset:1px;} .reset-button{min-height:42px; padding:9px 16px; border:1px solid #d1d5db; border-radius:8px; background:#fff; color:#374151;} .result-count{margin:0 0 16px; color:#6b7280;} .empty-state{padding:36px 20px; border:1px dashed #c4b5fd; border-radius:12px; background:#fff; text-align:center;} @media (max-width: 640px){.featured-product{grid-template-columns:64px minmax(0, 1fr); padding:16px;} .featured-product div[role="img"]{height:64px; grid-row:1 / span 2;} .featured-product h3{grid-column:2; grid-row:1;} .featured-product p{grid-column:2; grid-row:2;} .featured-product button{grid-column:1 / -1;} .filter-panel{grid-template-columns:1fr;}} .cart-section{margin-top:48px;} .cart-list{margin:0; padding:0; list-style:none;} .cart-item{display:flex; align-items:center; justify-content:space-between; gap:16px; padding:16px 20px; border:1px solid #e5e7eb; border-radius:12px; background:#fff;} .cart-item + .cart-item{margin-top:10px;} .cart-item p{margin:6px 0 0; color:#6b7280;} .quantity-controls{display:flex; align-items:center; gap:12px; white-space:nowrap;} .quantity-controls button{width:36px; height:36px; border:1px solid #d1d5db; border-radius:8px; background:#fff;} .cart-total{margin:18px 0 0; text-align:right; font-size:20px; font-weight:700;} @media (max-width: 640px){.cart-item{align-items:flex-start; flex-direction:column;}} .order-section{margin-top:48px;} .order-layout{display:grid; grid-template-columns:repeat(2, minmax(0, 1fr)); gap:20px;} .order-form, .order-summary{padding:24px; border:1px solid #e5e7eb; border-radius:16px; background:#fff;} .order-form label{display:block; margin:14px 0 6px; font-weight:700;} .order-form input{width:100%; padding:10px; border:1px solid #d1d5db; border-radius:8px;} .order-form input[aria-invalid="true"]{border-color:#dc2626;} .order-form button{margin-top:20px; padding:11px 18px; border:0; border-radius:9px; background:#6d28d9; color:#fff;} .form-error{color:#b91c1c;} .form-success{color:#166534;} .order-summary ul{padding:0; list-style:none;} .order-summary li{display:flex; justify-content:space-between; gap:16px; padding:10px 0; border-bottom:1px solid #e5e7eb;} .order-total{text-align:right; font-weight:700;} @media (max-width: 640px){.order-layout{grid-template-columns:1fr;}} .api-status{margin:16px 0 24px; padding:16px 20px; border-radius:12px; background:#f5f3ff; color:#4c1d95;} .api-error{background:#fef2f2; color:#b91c1c;}
```

학습 안내

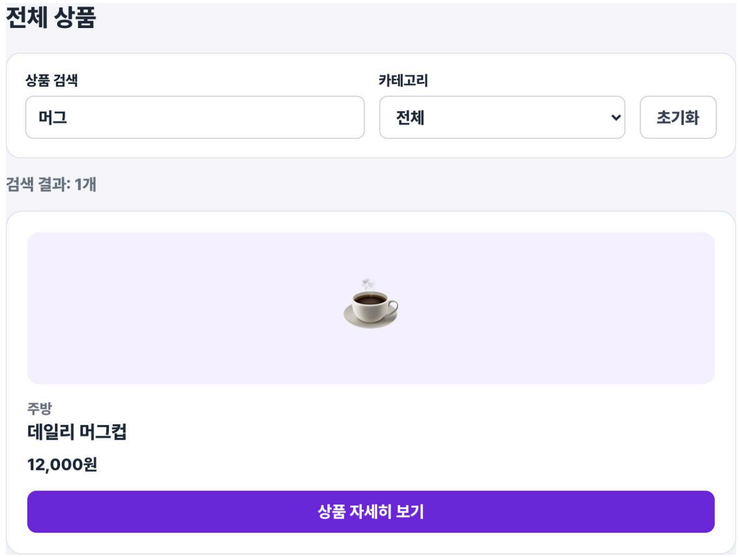
1일차 프로젝트 실행하기

터미널

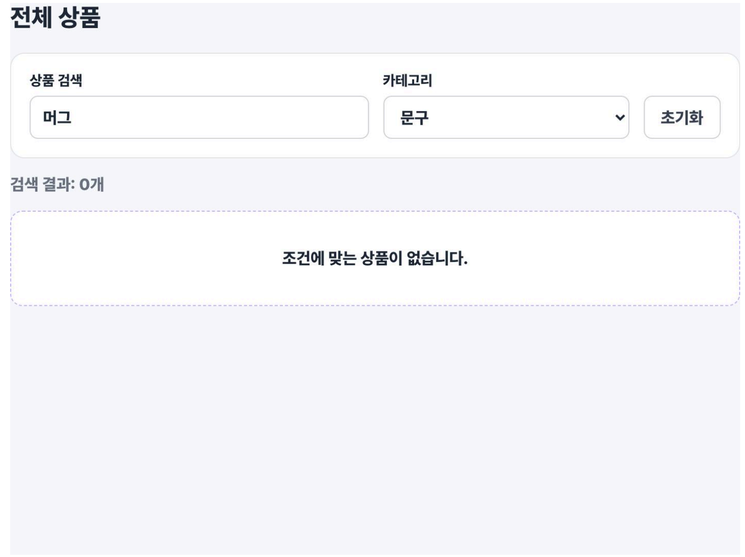
```
npm i
npm run dev
```



검색어 없음, 전체
상품 3개



머그 검색, 전체
머그컵 1개



머그 검색, 문구
검색 결과 0개

1교시
장바구니 담기와 수량과 합계

미션 - 장바구니에 상품 담고 수량 바꾸기

배울 내용

State 공유, 함수 Props, 배열과 객체 State 업데이트, 총 수량과 합계 계산

장바구니 State 추가

장바구니 초기 상태

초깃값은 빈 배열([])
아직 상품을 담지 않은 상태

src/App.jsx

밝은 줄만 추가!

```
function App() {
  const shopName = "오늘의 상점";
  const productName = "캔버스 토트백";
  const price = 18000;

  const [searchTerm, setSearchTerm] = useState("");
  const [selectedCategory, setSelectedCategory] = useState("전체");
  const [cartItems, setCartItems] = useState([]);

  const filteredProducts = products.filter((product) => {
    const matchesSearch = product.name
```

상품 담기 함수 추가

src/App.jsx

App 함수 내부: resetFilters 아래, return 앞에 함수 전체 추가

```
function addToCart(productId) {
  setCartItems((currentItems) => {
    const existingItem = currentItems.find(
      (item) => item.productId === productId,
    );
    if (existingItem) {
      return currentItems.map((item) =>
        item.productId === productId
          ? { ...item, quantity: item.quantity + 1 }
          : item,
      );
    }
    return [...currentItems, { productId, quantity: 1 }];
  });
}
```

함수 Props로 담기 버튼 연결

클릭 연결

App에서 함수 전달 → 카드에서 호출 → App State 갱신

성공 기준

담기 클릭 후 Components에서 cartItems의 ID와 수량 확인

src/App.jsx

상품 목록의 ProductCard 호출 전체 교체

```
<ProductCard
  key={product.id}
  product={product}
  onAddToCart={addToCart}
/>
```

src/components/ProductCard.jsx

기존 함수 선언 첫 줄만 교체, 밝은 줄만 추가/교체

```
function ProductCard({ product, onAddToCart }) {
    return (
        <article className="product-card">
```

src/components/ProductCard.jsx

기존 상품 자세히 보기 button 전체 교체

```
<button type="button" onClick={() => onAddToCart(product.id)}>
  장바구니에 담기
</button>
```

장바구니 상태를 공유하는 흐름

App

cartItems에는 상품 ID와 수량 저장
상품 이름과 가격은 products에서 조회

↓ **onAddToCart 함수 Props**

ProductCard

캔버스 토트백

18,000원

1 장바구니에 담기

`onAddToCart(1)`

함수 호출 →

2 addToCart 실행

상품 ID 1의 수량 증가

3 cartItems 변경

담기 전

```
[{productId: 1, quantity: 1}]
```

↓ `quantity + 1`

담기 후

```
[{productId: 1, quantity: 2}]
```

화면 계산 →

4 장바구니 (2개)

캔버스 토트백

수량 2개

합계

36,000원

18,000원 × 2개

실습 결과: 이전 State를 이용해 다음 값 계산

01 현재 값

currentItems는 해당 갱신 직전의 장바구니 배열
e.g. 토트백 1개

02 함수형 갱신

setCartItems(currentItems => 다음 배열)
e.g. 토트백 1개에 1개를 더해 2개로 변경

03 새 상품과 기존 상품

find로 존재 확인 → 새 상품은 추가 / 기존 상품은 수량 증가
e.g. 처음 담는 머그컵은 추가, 이미 담은 토트백은 1개 → 2개

실습 결과: 배열과 객체를 복사해 변경하기

01 배열 직접 변경 피하기

push와 item.quantity++는 기존 상태를 직접 수정
e.g. 토트백 수량을 item.quantity++로 직접 늘리는 방식

02 새 배열 만들기

spread로 추가, map으로 항목 갱신, filter로 제거
e.g. [...currentItems, { productId: 2, quantity: 1 }]로 머그컵 추가

03 새 객체 만들기

기존 객체를 복사한 뒤 quantity만 변경
e.g. 기존 토트백 수량 1은 두고, 수량 2인 새 객체 생성

Tip 새 배열을 전달하는 이유

같은 배열을 직접 바꾸면 React가 변경을 알아채지 못할 수 있음. 새 배열을 만들어 setter에 전달하면 변경을 구분할 수 있고, 이전 State도 보존됨.

수량 감소 함수 추가

src/App.jsx

App 함수 내부: addToCart 아래, return 앞에 추가

```
function decreaseQuantity(productId) {
  setCartItems((currentItems) =>
    currentItems
      .map((item) =>
        item.productId === productId
          ? { ...item, quantity: item.quantity - 1 }
          : item,
      )
      .filter((item) => item.quantity > 0),
  );
}
```

총 수량과 합계 계산

src/App.jsx

App 함수 내부: decreaseQuantity 아래, return 앞에 추가

```
const cartCount = cartItems.reduce(
  (count, item) => count + item.quantity,
  0,
);
const cartTotal = cartItems.reduce((total, item) => {
  const product = products.find((entry) => entry.id === item.productId);
  return total + product.price * item.quantity;
}, 0);
```

실습 결과: State와 계산값

State에는 상품 ID와 수량 저장, 총 수량과 합계는 렌더링 중 계산

cartItems (State)

상품 ID 1: 수량 2개
상품 ID 2: 수량 1개

이름과 가격은
products에서 조회

토트백 18,000원
머그컵 12,000원

→

reduce로 순서대로 합산

`cartCount`

0 → 2 → 3

토트백 2개 + 머그컵 1개

`cartTotal`

0 → 36,000 → 48,000

`18,000 × 2 + 12,000 × 1`

→

화면에 표시

총 수량 (cartCount)

3개

합계 (cartTotal)

48,000원

장바구니 영역 추가 (1/3)

src/App.jsx

전체 상품 section 뒤, 마지막 </main> 앞에 추가, 1/3 순서로 연결 후 한 번 저장

```
<section className="cart-section" aria-labelledby="cart-title">
  <h2 id="cart-title">장바구니 ({cartCount}개)</h2>
  {cartItems.length === 0 ? (
    <p className="empty-state">아직 담은 상품이 없습니다.</p>
  ) : (
    <>
      <ul className="cart-list">
        {cartItems.map((item) => {
          const product = products.find(
            (entry) => entry.id === item.productId,
          );
          return (
            <li className="cart-item" key={item.productId}>
              <div>
```

장바구니 영역 추가 (2/3)

src/App.jsx

전체 상품 section 뒤, 마지막 </main> 앞에 추가, 2/3 순서로 연결 후 한 번 저장

```
              <strong>{product.name}</strong>
              <p>{product.price.toLocaleString()}원 × {item.quantity}</p>
            </div>
            <div className="quantity-controls">
              <button
                type="button"
                aria-label={`${product.name} 수량 줄이기`}
                onClick={() => decreaseQuantity(product.id)}
              >
                -
              </button>
              <span>{item.quantity}</span>
              <button
                type="button"
```

장바구니 영역 추가 (3/3)

src/App.jsx

전체 상품 section 뒤, 마지막 </main> 앞에 추가, 3/3 순서로 연결 후 한 번 저장

```
                    aria-label={`${product.name} 수량 늘리기`}
                    onClick={() => addToCart(product.id)}
                  >
                    +
                  </button>
                </div>
              </li>
            );
          })}
        </ul>
        <p className="cart-total">합계: {cartTotal.toLocaleString()}원</p>
      </>
    )}
</section>
```

응용 실습 1: 상품 한 종류 삭제

수량과 관계없이 선택한 상품 한 종류를 바로 제거

성공 기준: 토트백 2개, 머그컵 1개에서 토트백 삭제 → 1개, 12,000원

힌트

1. 상품 이름보다 productId로 선택한 항목 식별
2. currentItems.filter로 해당 ID를 제외한 새 배열 반환
3. cartItems.map 내부, 상품별 quantity-controls에 배치

선택 심화

전체 비우기 버튼 추가 → 빈 장바구니 안내와 0개 표시

응용 실습 1: 삭제 함수 정답

src/App.jsx

App 함수 내부: decreaseQuantity 아래에 추가

```
function removeFromCart(productId) {
  setCartItems((currentItems) =>
    currentItems.filter((item) => item.productId !== productId),
  );
}
```

응용 실습 1 정답: 삭제와 전체 비우기

src/App.jsx

cartItems.map의 quantity-controls 안에 추가

```
<button type="button" onClick={() => removeFromCart(product.id)}>
  삭제
</button>
```

src/App.jsx

cart-title의 h2 바로 아래에 추가, 선택 심화

```
<button type="button" onClick={() => setCartItems([])}>
  장바구니 비우기
</button>
```

1교시 핵심: 장바구니 상태와 화면

핵심	현재 코드	기억할 내용
상태의 위치	App의 cartItems	상품 ID와 수량을 부모에서 함께 관리
클릭 연결	onAddToCart → addToCart	카드는 전달받은 함수를 호출, App이 State 변경
수량 변경과 삭제	map, filter, spread	이전 State를 이용해 새 배열과 객체 생성
화면 계산	cartCount, cartTotal	총 수량과 가격 × 수량 합계를 현재 배열에서 계산

동작 연결

토트백 2개 + 머그컵 1개 = 총 3개, 48,000원
검색 조건 변경은 상품 목록만 변경, 장바구니는 유지

2교시
주문서 입력과 DB 준비

미션 - 주문자 정보 입력하고 오류 확인하기

배울 내용

입력값 관리, 제출 시 입력 검증, PostgreSQL 상품 테이블

주문 입력 상태 추가

주문서 초기 상태

name과 email은 빈 문자열, 처음에는 오류 문구 숨김

src/App.jsx

App 함수 내부: cartItems 상태 바로 아래에 추가

```
const [orderForm, setOrderForm] = useState({
  name: "",
  email: "",
});
const [wasSubmitted, setWasSubmitted] = useState(false);
```

입력 변경과 제출 연결

입력과 제출 처리

변경한 필드만 새 객체에 반영, 제출 시 페이지 이동 방지

src/App.jsx

App 함수 내부: decreaseQuantity 아래, cartCount 앞에 추가

```
function handleOrderChange(event) {
  const { name, value } = event.target;
  setOrderForm((currentForm) => ({ ...currentForm, [name]: value }));
}
function handleOrderSubmit(event) {
  event.preventDefault();
  setWasSubmitted(true);
}
```

입력 오류와 유효성 계산

검증할 항목

공백 이름, 잘못된 이메일,
빈 장바구니를 오류로 판정

src/App.jsx

App 함수 내부: cartTotal 아래, return 앞에 추가

```
const orderErrors = {
  name: orderForm.name.trim() ? "" : "이름을 입력해 주세요.",
  email: /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(orderForm.email.trim())
    ? ""
    : "올바른 이메일을 입력해 주세요.",
  cart: cartCount > 0 ? "" : "장바구니에 상품을 담아 주세요.",
};
const isOrderValid = Object.values(orderErrors).every((message) => !
message);
```

주문서와 요약 영역 추가 (1/4)

src/App.jsx

마지막 </main>을 연결한 전체 코드로 교체, 기존 장바구니 유지, 1/4 순서로 연결 후 한 번 저장

```
<section className="order-section" aria-labelledby="order-title">
    <h2 id="order-title">주문서 미리 보기</h2>
    <div className="order-layout">
      <form className="order-form" noValidate onSubmit={handleOrderSubmit}>
        <h3>주문자 정보</h3>
        <label htmlFor="order-name">이름</label>
        <input
          id="order-name"
          name="name"
          value={orderForm.name}
          onChange={handleOrderChange}
          aria-invalid={wasSubmitted && Boolean(orderErrors.name)}
        />
        {wasSubmitted && orderErrors.name && (
          <p className="form-error" role="alert">{orderErrors.name}</p>
        )}
        <label htmlFor="order-email">이메일</label>
```

주문서와 요약 영역 추가 (2/4)

src/App.jsx

마지막 </main>을 연결한 전체 코드로 교체, 기존 장바구니 유지, 2/4 순서로 연결 후 한 번 저장

```
        <input
          id="order-email"
          name="email"
          type="email"
          value={orderForm.email}
          onChange={handleOrderChange}
          aria-invalid={wasSubmitted && Boolean(orderErrors.email)}
        />
        {wasSubmitted && orderErrors.email && (
          <p className="form-error" role="alert">{orderErrors.email}</p>
        )}
        {wasSubmitted && orderErrors.cart && (
          <p className="form-error" role="alert">{orderErrors.cart}</p>
        )}
        <button type="submit">입력 확인</button>
        {wasSubmitted && isOrderValid && (
          <p className="form-success" role="status">
```

주문서와 요약 영역 추가 (3/4)

src/App.jsx

마지막 </main>을 연결한 전체 코드로 교체, 기존 장바구니 유지, 3/4 순서로 연결 후 한 번 저장

```
            입력 확인 완료! 실제 주문 저장은 9차시에 연결합니다.
          </p>
        )}
      </form>
      <div className="order-summary">
        <h3>주문 요약</h3>
        {cartItems.length === 0 ? (
          <p>담은 상품이 없습니다.</p>
        ) : (
          <ul>
            {cartItems.map((item) => {
              const product = products.find(
                (entry) => entry.id === item.productId,
              );
              return (
                <li key={item.productId}>
                  <span>{product.name} × {item.quantity}</span>
```

주문서와 요약 영역 추가 (4/4)

src/App.jsx

마지막 </main>을 연결한 전체 코드로 교체, 기존 장바구니 유지, 4/4 순서로 연결 후 한 번 저장

```
                  <span>
                    {(product.price * item.quantity).toLocaleString()}원
                  </span>
                </li>
              );
            })}
          </ul>
        )}
        <p className="order-total">
          상품 {cartCount}개 · 합계 {cartTotal.toLocaleString()}원
        </p>
      </div>
    </div>
  </section>
</main>
```

입력창과 orderForm의 연결

화면의 입력창

이메일

name="email"
student@example.com

입력 변경 → onChange 실행
event.target.name은 email
event.target.value는 입력한 문자열

onChange
↔
value

orderForm (State)

```
setOrderForm((currentForm) => ({
  ...currentForm,
  [name]: value,
}));
```

[name]으로 email 필드 선택
spread로 기존 name 필드 보존
바뀐 orderForm.email을 value로 다시 표시

폼 제출과 preventDefault

브라우저의 기본 폼 제출

버튼 클릭 또는 Enter → submit 이벤트
브라우저가 form의 action 주소로 전송하고 페이지 이동
action이 없으면 현재 문서 주소로 제출

event.preventDefault()

```
function handleOrderSubmit(event) {
  event.preventDefault();
  setWasSubmitted(true);
}
```

브라우저의 자동 전송과 페이지 이동 취소
현재 화면과 입력 State를 유지하며 검증 결과 표시
onSubmit 함수의 나머지 코드는 계속 실행

현재 실습의 제출 결과

wasSubmitted로 오류 표시 시점 관리, 오류는 현재 입력에서 계산
주문자 입력과 장바구니 확인까지 구현, 서버 저장은 별도 기능

응용 실습 2: 이메일 확인 입력

emailConfirm 입력 추가 → 이메일 두 값의 일치 검사

성공 기준: 불일치 시 오류, 일치하고 나머지 조건도 만족하면 입력 확인 완료

힌트

1. orderForm에 emailConfirm: "" 추가
2. name="emailConfirm"과 기존 handleOrderChange 재사용
3. orderErrors에 두 이메일의 trim 값 비교 추가

선택 심화

제출 후 한쪽 이메일 수정 → 안내가 현재 입력에 맞게 갱신

응용 실습 2 정답: 상태와 검증

src/App.jsx

기존 orderForm 상태 선언 전체 교체

```
const [orderForm, setOrderForm] = useState({
  name: "",
  email: "",
  emailConfirm: "",
});
```

src/App.jsx

기존 orderErrors와 isOrderValid 영역 전체 교체

```
const orderErrors = {
    name: orderForm.name.trim() ? "" : "이름을 입력해 주세요.",
    email: /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(orderForm.email.trim())
      ? ""
      : "올바른 이메일을 입력해 주세요.",
    emailConfirm: orderForm.emailConfirm.trim() === orderForm.email.trim()
    ? "" : "이메일이 일치하지 않습니다.",
  cart: cartCount > 0 ? "" : "장바구니에 상품을 담아 주세요.",
  };
  const isOrderValid = Object.values(orderErrors).every((message) => !message);
```

응용 실습 2: 확인 입력 정답

src/App.jsx

주문서의 이메일 오류 영역 다음, 장바구니 오류 영역 앞에 추가

```
<label htmlFor="order-email-confirm">이메일 확인</label>
<input
  id="order-email-confirm"
  name="emailConfirm"
  type="email"
  value={orderForm.emailConfirm}
  onChange={handleOrderChange}
  aria-invalid={wasSubmitted && Boolean(orderErrors.emailConfirm)}
/>
{wasSubmitted && orderErrors.emailConfirm && (
  <p className="form-error" role="alert">{orderErrors.emailConfirm}</p>
)}
```

주문서에서 기억할 값과 계산할 값

기억할 값

orderForm
이름과 이메일 입력값

wasSubmitted
제출 버튼을 눌렀는지 여부

cartItems
담은 상품과 수량

현재 값 사용 →

렌더링 중 계산

orderErrors

공백 이름, 이메일 형식, 빈 장바구니 검사

isOrderValid

오류 유무에서 유효성 계산

주문 요약

현재 상품과 수량에서 목록과 합계 계산

→

화면 반영

제출 전
오류 문구 숨김

제출 후
현재 오류 표시

입력 수정
검증과 요약 재계산

DB 준비: 접속 확인과 새 shop DB

먼저 확인할 정보

서버 실행

Docker Desktop에서 PostgreSQL Running 확인

DBeaver 연결

기존 연결의 사용자, 비밀번호, 호스트 포트 확인
5432는 예시, 실제 연결값 사용

새 DB 이름

shop이 이미 있으면 다른 새 이름 사용
생성한 DB 이름을 .env에도 동일하게 기록

기존 PostgreSQL 서버

기존 DB

기존 수업의 테이블과 데이터를 유지

새 shop DB

이번 쇼핑몰의 products 테이블 생성

서버를 새로 설치하지 않고, 같은 서버 안에 새 데이터베이스 생성

새 shop 데이터베이스 생성

생성한 DB 확인

DB 목록 새로고침 → shop 연결로 변경 후 다음 SQL 실행

DBeaver: 기존 postgres DB의 SQL 편집기

자동 커밋 상태에서 실행, 트랜잭션 묶음 안에서는 실행 불가

```
CREATE DATABASE shop;
```

상품 테이블 생성

상품 테이블 조건

product_id 자동 생성, 가격과 재고는 0 이상, 필수값 NULL 방지

DBeaver: shop DB의 SQL 편집기

새 shop의 public 스키마에서 한 번 실행

```
CREATE TABLE public.products (
  product_id INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  name TEXT NOT NULL,
  category TEXT NOT NULL,
  price NUMERIC(10, 2) NOT NULL CHECK (price >= 0),
  stock_quantity INTEGER NOT NULL CHECK (stock_quantity >= 0)
);
```

실습 상품 세 개 입력

기본 상품

토트백 18,000원 / 머그컵 12,000원 / 노트 4,500원

DBeaver: shop DB의 SQL 편집기

상품 테이블 생성 다음 실행, 기존 이름은 중복 삽입 방지

```
INSERT INTO public.products (name, category, price, stock_quantity)
SELECT sample.name, sample.category, sample.price,
sample.stock_quantity
FROM (VALUES
  ('캔버스 토트백', '가방', 18000, 10),
  ('데일리 머그컵', '주방', 12000, 15),
  ('포켓 노트', '문구', 4500, 20)
) AS sample(name, category, price, stock_quantity)
WHERE NOT EXISTS (
  SELECT 1 FROM public.products AS p WHERE p.name = sample.name
);
```

입력한 상품 확인

조회 결과

상품 3행 확인, 실제 product_id는 1, 2, 3과 다를 수 있음

DBeaver: shop DB의 SQL 편집기

새로 만든 shop 연결에서 실행

```sql
SELECT product_id, name, category, price, stock_quantity
FROM public.products
ORDER BY product_id;
```

Python 가상환경과 패키지 준비

터미널: Windows CMD

프로젝트 폴더에서 실행, Python 3.13 사전 설치

```
py -3.13 -m venv backend\.venv
backend\.venv\Scripts\python.exe -m pip install fastapi uvicorn "psycopg[binary]" python-dotenv
```

DB 접속 정보 파일 만들기

접속 정보 파일

backend 폴더에 .env 생성
DB 이름은 생성한 데이터베이스와 동일하게 지정

backend/.env

새 파일 생성, 계정, 비밀번호, 포트는 DBeaver 실제 연결값으로 변경

```
SHOP_DB_HOST=127.0.0.1
SHOP_DB_PORT=5432
SHOP_DB_NAME=shop
SHOP_DB_USER=postgres
SHOP_DB_PASSWORD=본인의_DB_비밀번호
```

비밀번호와 가상환경을 Git에서 제외

Git 제외 대상

.env에는 DB 접속 비밀번호 포함
Git에 올리면 저장소 접근자에게 노출되고, 커밋 이력에도 남을 수 있음
.venv는 설치 환경이므로 requirements.txt로 다시 생성

프로젝트 루트 .gitignore

파일 맨 끝에 추가, 이미 있으면 중복 추가 생략

```
backend/.env
backend/.venv/
```

3교시
FastAPI 상품 API 만들기

미션 - DB의 상품을 조회하는 API 만들기

배울 내용

HTTP 메서드와 상태 코드, DB 연결, JSON 응답

HTTP 메서드와 CRUD

요청에 담는 목적

GET	데이터 조회
POST	처리 요청 또는 새 데이터 생성
PUT	대상 전체 교체
PATCH	일부 필드 수정
DELETE	대상 삭제

데이터 작업의 네 가지 역할

C Create	생성	POST
R Read	조회	GET
U Update	변경	PUT / PATCH
D Delete	삭제	DELETE

현재 실습

GET /products → R (Read)
GET /health → 서버 실행 확인

FastAPI 시작 파일 만들기

서버 상태 확인

/health는 API 실행 확인, DB 연결 검사는 다음 /products에서 수행

backend/main.py

새 파일 생성, 이번 단계는 기존 backend가 없는 1일차 코드 기준

```
from pathlib import Path
from fastapi import FastAPI
from dotenv import load_dotenv
load_dotenv(Path(__file__).with_name(".env"))
app = FastAPI()
@app.get("/health")
def health():
    return {"status": "ok"}
```

FastAPI 서버 실행

접속 주소

127.0.0.1:8000/health →
{"status":"ok"}, /docs 문서 확인

터미널: Windows CMD

프로젝트 폴더에서 실행, 이 터미널은 켜 둠

```
backend\.venv\Scripts\python.exe -m uvicorn backend.main:app --reload
```

HTTP 상태 코드로 응답 확인

상태 코드	요청 사례	의미
200 OK	GET /products	상품 배열을 정상 응답
404 Not Found	GET /없는경로	요청한 경로가 없음
405 Method Not Allowed	POST /products	경로는 있으나 POST를 지원하지 않음
500 Internal Server Error	GET /products 중 처리 실패	DB 연결이나 서버 코드 오류로 응답 실패

연결 실패는 HTTP 500과 다름

서버가 꺼져 있거나 주소가 잘못되면 HTTP 응답을 받지 못함
상태 코드는 요청을 받은 서버가 보낸 응답의 결과임

DB 조회 import 추가

패키지의 역할

os: 환경변수에서 DB 접속값 읽기

psycopg: PostgreSQL 연결,
SQL 실행과 조회 결과 받기

dict_row: 조회 행을 딕셔너리로 변환
row["name"]으로 열 이름 사용

backend/main.py

파일 맨 위 import 모음에 추가

```
import os
import psycopg
from psycopg.rows import dict_row
```

DB 접속 함수 추가

DB 접속 설정

SHOP_DB_* 값으로 접속,
사용자와 비밀번호는 필수

backend/main.py

health 함수 아래에 함수 전체 추가

```
def get_connection():
    return psycopg.connect(
        host=os.getenv("SHOP_DB_HOST", "127.0.0.1"),
        port=int(os.getenv("SHOP_DB_PORT", "5432")),
        dbname=os.getenv("SHOP_DB_NAME", "shop"),
        user=os.environ["SHOP_DB_USER"],
        password=os.environ["SHOP_DB_PASSWORD"],
        row_factory=dict_row,
    )
```

상품 조회 경로 추가 (1/2)

backend/main.py

get_connection 아래에 연결한 함수 전체 추가, 1/2 순서로 연결 후 한 번 저장

```
@app.get("/products")
def list_products():
    with get_connection() as connection:
        with connection.cursor() as cursor:
            cursor.execute(
                """
                SELECT product_id, name, category, price, stock_quantity
                FROM public.products
                ORDER BY product_id
                """
            )
            rows = cursor.fetchall()
```

상품 조회 경로 추가 (2/2)

backend/main.py

get_connection 아래에 연결한 함수 전체 추가, 2/2 순서로 연결 후 한 번 저장

```
    emoji_by_category = {"가방": "👜", "주방": "☕", "문구": "📒"}
    return [
        {
            "id": row["product_id"],
            "name": row["name"],
            "category": row["category"],
            "price": float(row["price"]),
            "stock_quantity": row["stock_quantity"],
            "emoji": emoji_by_category.get(row["category"], "🛍️"),
        }
        for row in rows
    ]
```

상품 조회에서 각 기술의 역할

React와 브라우저

사용자에게 상품 표시

GET /products로 상품 요청
응답 배열로 카드와 검색 결과 구성

FastAPI

요청과 데이터 변환

요청을 받고 DB에 SQL 실행
행 데이터를 JSON 배열로 변환

PostgreSQL

상품 데이터 보관

상품 행과 열 저장
ID 자동 생성, 가격과 재고 조건 관리

DB 값	API 응답	화면에서 사용
DB product_id	응답 id	React에서 실제 상품 ID 사용
DB price	숫자 price	가격과 수량 곱셈에 사용
DB category	emoji 추가	카테고리별 이모지, 기본값 사용

응용 실습 3: 새 카테고리 상품 추가

생활 카테고리의 미니 파우치 8,000원, 재고 5개 추가

성공 기준: SELECT 4행 → /products 응답 4개, 새 ID와 기본 이모지 확인

힌트

1. DBeaver의 shop DB, public.products
2. name, category, price, stock_quantity 입력, ID는 DB에서 자동 생성
3. 브라우저 /products 새로고침, React는 아직 임시 배열

선택 심화

음수 가격과 재고 입력 시 CHECK 제약 오류 확인

응용 실습 3: 상품 추가 정답

DBeaver: shop DB의 SQL 편집기

추가 상품은 한 번만 입력

```
INSERT INTO public.products (name, category, price, stock_quantity)
VALUES ('미니 파우치', '생활', 8000, 5);
SELECT product_id, name, category, price, stock_quantity
FROM public.products
ORDER BY product_id;
```

응용 실습 3: 제약조건 확인

DBeaver: shop DB의 SQL 편집기

선택 심화, 가격을 -100으로 입력해 의도적인 오류 관찰

```
INSERT INTO public.products (name, category, price, stock_quantity)
VALUES ('가격 오류 상품', '실습', -100, 5);
```

실습 검증: API 응답과 DB 연결

확인 대상	실행할 동작	기대 결과 또는 점검
서버 실행	/health 열기	{"status":"ok"} 응답
상품 조회	/products 열기	상품 배열, 실제 id, 숫자 price 확인
DB 연결 오류	/health 성공, /products 실패	컨테이너, 포트, DB 이름, .env 계정 확인
실습 완료	상품 추가 후 /products 새로고침	상품 4개, 새 ID와 기본 이모지 확인

4교시
React에서 DB 상품 표시하기

미션 - API의 상품을 React 화면에 표시하기

배울 내용

Hooks: Promise, 비동기 요청, useEffect, 로딩, 오류 표시

일반 변수와 State의 차이

useState는 파일 상단에서 import

일반 지역 변수

```
function Counter() {
  let count = 0;
  function increment() {
    count += 1;
  }
  return (
    <button onClick={increment}>
      {count}
    </button>
  );
}
```

변경만으로 React의 화면 갱신 요청 없음
컴포넌트가 다시 실행되면 지역 변수 재생성

useState

```
function Counter() {
  const [count, setCount] = useState(0);
  function increment() {
    setCount(current => current + 1);
  }
  return (
    <button onClick={increment}>
      {count}
    </button>
  );
}
```

React가 렌더링 사이의 값 보존
setter 호출 → 렌더링 요청 → 새 값의 UI 계산

Hooks: 값을 기억하고 API에 연결하기

Hook은 컴포넌트에서 React의 기능을 사용할 수 있게 연결하는 함수임

useState 값 기억과 화면 갱신

버튼 클릭
setCartItems 호출

→

React가 값 기억
다음 렌더링에서도
새 cartItems 사용

→

화면 다시 계산
새 수량과 합계 표시

useEffect 화면과 외부 시스템 연결

화면에 연결
Effect의 상품 요청 실행

→

외부 API
상품 응답을 받고
setProducts(data) 호출

→

받은 상품 표시
products로 카드 계산

Hook 호출 위치와 코드 역할

App 컴포넌트 최상위

```
function App() {
  const [cartItems, setCartItems] = useState([]);
  useEffect(() => {
    // 상품 API 연결
  }, []);

  function addToCart(productId) {
    // 클릭 시 State 변경
  }

  const cartCount = cartItems.reduce(
    (count, item) => count + item.quantity, 0,
  );
  return <main>...</main>;
}
```

Hook 호출

useState와 useEffect는 함수 최상위에서 호출
조건문, 반복문, 이벤트 함수 내부 호출 금지

이벤트

사용자의 담기 클릭을 addToCart에서 처리

렌더링 계산

검색 결과와 합계는 현재 배열에서 바로 계산
값 계산만을 위해 Effect를 만들 필요 없음

커스텀 Hook에서도 동일한 호출 규칙 적용

React 출처를 서버에서 허용하기

CORS

브라우저가 다른 출처의
응답을 읽을 수 있는 조건

출처

프로토콜, 호스트, 포트를 실제 React
주소와 일치시킴

backend/main.py

파일 맨 위 import 모음에 추가, 밝은 줄만 추가/교체

```
from fastapi.middleware.cors import CORSMiddleware

import os
import psycopg
from psycopg.rows import dict_row
```

backend/main.py

app = FastAPI() 바로 아래에 추가

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173", "http://127.0.0.1:5173"],
    allow_credentials=True,
    allow_methods=["GET", "POST"],
    allow_headers=["Authorization", "Content-Type"],
)
```

URL 구조와 CORS 출처 맞추기

출처 (Origin)

`http://localhost:5173`

프로토콜	호스트	포트
http	localhost	5173

경로

`/products`

경로, 쿼리, 프래그먼트는
출처에 포함되지 않음

비교 주소	기준 주소와 비교	달라지는 부분
http://localhost:5173/cart	같은 출처	경로만 변경
http://127.0.0.1:5173	다른 출처	호스트 변경
http://localhost:8000	다른 출처	포트 변경
https://localhost:5173	다른 출처	프로토콜 변경

allow_origins에는 실제 React 주소의 프로토콜, 호스트, 포트를 모두 일치시켜 등록
React(5173)와 API(8000)는 서로 다른 출처, 서버가 React 출처를 허용해야 브라우저가 응답을 읽을 수 있음

조회되지 않은 상품의 참조 방지

합계

products.find가 undefined이면
가격 접근 대신 0을 반환

표시

상품이 없으면 행 표시 생략
HTTP 오류 처리와 구분되는
방어 코드

src/App.jsx

cartTotal의 기존 return 한 줄 교체, 밝은 줄만 추가/교체

```
const cartTotal = cartItems.reduce((total, item) => {
  const product = products.find((entry) => entry.id ===
item.productId);
  return total + (product?.price ?? 0) * item.quantity;
}, 0);
```

src/App.jsx

장바구니와 주문 요약의 두 cartItems.map에 각각 추가, products.find 바로 뒤, 밝은 줄만 추가/교체

```
{cartItems.map((item) => {
  const product = products.find(
    (entry) => entry.id === item.productId,
  );
  if (!product) return null;
  return (
    <li key={item.productId}>
```

상품 요청 import와 State 준비

유지

검색, 카테고리, 장바구니, 주문 상태 유지

전환 순서

productName과 price는 추천 영역 교체 시 제거, 계산 코드는 이어서 적용

src/App.jsx

파일 맨 위 기존 React import 한 줄 교체, 밝은 줄만 추가/교체

```
import { useEffect, useState } from "react";
import "./App.css";
import ProductCard from "./components/ProductCard";
```

src/App.jsx

함수 밖 products와 categories 삭제 → App의 shopName 아래에 추가

```
const [products, setProducts] = useState([]);
const [isLoading, setIsLoading] = useState(true);
const [productError, setProductError] = useState("");
```

추천 상품과 카테고리 계산

추천 상품과 카테고리

첫 DB 상품은 추천 상품, Set으로 중복 카테고리 제거

src/App.jsx

App 함수 내부: wasSubmitted 선언 아래, filteredProducts 계산 앞에 추가

```
const featuredProduct = products[0];
const categories = [
  "전체",
  ...new Set(products.map((product) =>
product.category).filter(Boolean)),
];
```

Promise: 대기에서 성공 또는 실패로

Promise는 나중에 도착할 비동기 결과를 다루는 객체, ES2015(ES6)에 도입

대기 (pending)

요청 진행 중
성공과 실패 결과 미확정

성공 (fulfilled)

resolve(결과값) → 성공값 전달
then에서 결과값 사용

실패 (rejected)

reject(오류) → 실패 전달
catch에서 오류 처리

resolve()는 결과를 전달하는 함수, fulfilled는 성공 상태

다른 Promise로 resolve하면 그 결과를 기다림. resolved와 fulfilled는 항상 같은 뜻이 아님.

중첩 콜백과 Promise 체이닝

콜백 중첩

```
request((response) => {
  readBody(response, (data) => {
    applyResult(data);
  }, handleError);
}, handleError);
```

다음 작업이 콜백 안쪽에 중첩
작업별 오류 처리 위치 분산

Promise 체이닝

```
request()
  .then((response) => {
    return readBody(response);
  })
  .then((data) => {
    applyResult(data);
  })
  .catch(handleError);
```

return한 Promise의 결과를 다음 then에 전달
거부, 처리 중 예외는 뒤의 catch로 전달

fetch의 Promise로 상품 응답 읽기

상품 응답 처리

```
fetch("http://127.0.0.1:8000/products")
  .then((response) => {
    if (!response.ok) {
      throw new Error(`상품 조회 실패 (${response.status})`);
    }
    return response.json();
  })
  .then((data) => {
    if (!Array.isArray(data)) {
      throw new Error("상품 응답 형식이 올바르지 않습니다.");
    }
    setProducts(data);
  })
  .catch((error) => {
    setProductError(error.message);
  });
```

fetch는 Promise 반환

요청 직후에는 결과가 미정
응답이 오면 Response 전달
response.json()도 Promise 반환

응답 처리 흐름

fetch
↓ Response
response.ok 확인
↓ response.json()
상품 배열
↓ setProducts(data)
화면 갱신

같은 Promise를 두 방식으로 처리

async/await는 Promise 처리를 순서대로 읽기 쉽게 작성하는 문법 (ES2017)

처리할 일	then / catch	async / await
응답 받기	fetch(url).then(response => …)	const response = await fetch(url)
본문 읽기	return response.json()	const data = await response.json()
화면 반영	.then(data => setProducts(data))	setProducts(data)
오류 처리	.catch(error => …)	try { … } catch (error) { … }

기다리는 범위

await는 현재 async 함수의 이어지는 실행을 기다림. 브라우저 전체가 멈추는 것은 아님.

의존성 배열에 따른 Effect 실행

의존성 배열	실행 시점	현재 코드와 연결
배열 생략	커밋 이후마다 실행	렌더링과 상태 갱신의 반복에 주의
빈 배열 []	첫 마운트 후 실행	현재 상품 요청은 검색 입력에 의존하지 않음
[값]	첫 마운트와 값 변경 후 실행	의존성 값이 변경되면 이전 Effect 정리 후 실행

배열을 정하는 기준

Effect가 사용하는 반응형 값을 의존성에 포함
실행 횟수를 줄이기 위해 필요한 값을 임의로 제외하지 않음

개발 StrictMode의 추가 setup과 cleanup은 의존성 배열과 별개의 점검 동작임

상품 요청 Effect 추가 (1/2)

src/App.jsx

App 함수 내부: categories 아래, filteredProducts 앞에 추가, 1/2 순서로 연결 후 한 번 저장

```
useEffect(() => {
  const controller = new AbortController();
  async function loadProducts() {
    try {
      const response = await fetch("http://127.0.0.1:8000/products", {
        signal: controller.signal,
      });
      if (!response.ok) {
        throw new Error(`상품 조회 실패 (${response.status})`);
      }
      const data = await response.json();
      if (!Array.isArray(data)) {
        throw new Error("상품 응답 형식이 올바르지 않습니다.");
      }
      setProducts(data);
```

상품 요청 Effect 추가 (2/2)

src/App.jsx

App 함수 내부: categories 아래, filteredProducts 앞에 추가, 2/2 순서로 연결 후 한 번 저장

```
      setCartItems([]);
      setProductError("");
    } catch (error) {
      if (error.name !== "AbortError") {
        setProductError(error.message);
      }
    } finally {
      if (!controller.signal.aborted) {
        setIsLoading(false);
      }
    }
  }
  loadProducts();
  return () => controller.abort();
}, []);
```

[]와 마운트

마운트는 컴포넌트가 처음 화면에 추가되는 시점임

빈 배열 []은 첫 마운트 후 Effect 실행
검색 입력으로 다시 렌더링돼도 요청을 반복하지 않음

개발 StrictMode에서는 정리 동작 점검을 위해 setup → cleanup → setup이 추가 실행될 수 있음

Effect 정리와 요청 취소

setup

Effect 실행
AbortController 생성
상품 API 요청 시작

정리 시점 →

cleanup

화면에서 제거되거나 재실행 전
controller.abort()로 요청 취소
AbortError는 조회 오류로 표시하지 않음

개발 StrictMode

setup → cleanup → setup을 추가 실행해 정리 동작 점검
Network에서 취소 요청과 최종 성공 응답을 구분, StrictMode 유지

추천 상품 영역 교체

src/App.jsx

productName과 price 두 선언 삭제 → featured-section의 기존 article 전체 교체

```
{featuredProduct ? (
  <article className="featured-product">
    <div role="img" aria-label={`${featuredProduct.name} 상품 이미지`}>
      {featuredProduct.emoji}
    </div>
    <h3>{featuredProduct.name}</h3>
    <p>{featuredProduct.price.toLocaleString()}원</p>
    <button type="button">상품 보러 가기</button>
  </article>
) : (
  <p className="empty-state">
    {isLoading ? "추천 상품을 불러오는 중입니다." : "추천 상품이 없습니다."}
  </p>
)}
```

상품 목록의 요청 상태 표시 (1/2)

src/App.jsx

전체 상품의 result-count부터 목록 조건부 JSX 끝까지 교체, filter-panel 유지, 1/2 순서로 연결 후 한 번 저장

```
{isLoading ? (
  <p className="api-status" role="status">상품을 불러오는 중입니다.</p>
) : productError ? (
  <p className="api-status api-error" role="alert">
    상품을 불러오지 못했습니다: {productError}
  </p>
) : (
  <>
    <p className="result-count">검색 결과: {filteredProducts.length}개</p>
    {filteredProducts.length > 0 ? (
      <div className="product-grid">
        {filteredProducts.map((product) => (
```

상품 목록의 요청 상태 표시 (2/2)

src/App.jsx

전체 상품의 result-count부터 목록 조건부 JSX 끝까지 교체, filter-panel 유지, 2/2 순서로 연결 후 한 번 저장

```
          <ProductCard
            key={product.id}
            product={product}
            onAddToCart={addToCart}
          />
        ))}
      </div>
    ) : (
      <p className="empty-state">조건에 맞는 상품이 없습니다.</p>
    )}
  </>
)}
```

실습 결과: 요청의 성공과 실패

01 **HTTP 오류**

fetch가 HTTP 오류를 자동 예외 처리하지 않으므로 response.ok 확인

02 **네트워크 오류**

서버 중지, 잘못된 주소, CORS 문제는 오류 안내와 Network에서 확인

03 **성공했지만 상품 0개**

정상 응답 []는 요청 실패와 다름, 상품 없음 화면 필요

API 요청과 화면 계산의 역할 구분

useEffect

컴포넌트와 상품 API 연결
마운트 후 상품 요청
응답을 products State에 반영

```
useEffect(() => {
  // API 요청과 정리
}, []);
```

렌더링 중 계산

products로 추천 상품과 카테고리 계산
검색 조건으로 filteredProducts 계산
cartItems로 수량과 합계 계산

현재 State에서 계산할 수 있는 값은
별도 State나 Effect로 복제하지 않음

실습 검증: 로딩과 오류, 기존 기능

검증 항목	실행할 동작	기대 결과
로딩	Network에서 느린 연결 적용 후 새로고침	대기 안내 후 상품 표시
DB 변경	상품 이름 수정, 커밋 후 새로고침	카드와 검색에 바뀐 이름 반영
연결 오류	FastAPI 중지 후 새로고침	상품 조회 오류 안내
복구	FastAPI 재시작 후 새로고침	상품 다시 표시
기존 기능	담기, 수량 변경, 주문 입력 실행	합계와 주문 요약 정상 계산

응용 실습 4: 두 종류의 빈 화면

DB 상품 0개와 검색 결과 0개의 안내를 구분

성공 기준: API [] → 등록된 상품 없음 / 상품이 있지만 검색 불일치 → 조건에 맞는 상품 없음

힌트

1. products.length는 API에서 받은 전체 상품 수
2. filteredProducts.length는 검색, 카테고리를 적용한 결과
3. 로딩 → 오류 → 성공 영역 안에서 전체 상품 수로 빈 안내 구분

선택 심화

생활 카테고리 표시 → 파우치 검색, 담기 → 토트백과 합계 검증

응용 실습 4: 빈 화면 정답

src/App.jsx

성공 영역의 기존 empty-state 한 곳만 교체

```
<p className="empty-state">
  {products.length === 0
    ? "등록된 상품이 없습니다."
    : "조건에 맞는 상품이 없습니다."}
</p>
```

참고 자료

React 공식 문서

react.dev/reference/react/hooks, react.dev/learn/synchronizing-with-effects

State와 배열

react.dev/learn/updating-arrays-in-state, react.dev/learn/queueing-a-series-of-state-updates

HTTP와 비동기

MDN HTTP Methods, Status / Using promises / Using Fetch

서버와 DB

fastapi.tiangolo.com/tutorial/cors/, psycopg.org/psycopg3/docs/, postgresql.org/docs/

원자료

사용자 첨부 원교안 4~7차시, 1일차 및 최종 완성 GitHub 코드

2일차 핵심: 입력에서 DB 상품까지

State로 기억

입력값, 제출 여부, 장바구니
API 상품, 로딩 여부, 요청 오류

현재 값에서 계산

검색 결과, 카테고리, 총 수량과 합계
입력 오류, 유효성, 주문 요약

외부 데이터 연결

PostgreSQL → FastAPI → fetch → products State → 상품 화면

완성한 기능

장바구니 담기와 수량 변경, 삭제
주문 입력 검증과 요약
DB 상품 조회, 로딩과 오류, 두 종류의 빈 화면